

Algoritmi e Strutture Dati – 02/02/2026

Esercizio A1 – Complessità – Punti ≥ 8

L'equazione di ricorrenza associata all'algoritmo è

$$T(n) = \begin{cases} 1 & n < 10 \\ 2T(n-10) + 1 & n \geq 10 \end{cases}$$

Utilizzando il Teorema delle Ricorrenze lineari di ordine costante, è possibile vedere che il costo della funzione è $T(n) = \Theta(2^n)$.

Errori ricorrenti Alcune soluzioni presentate dagli studenti contenevano questa ricorrenza:

$$T(n) = \begin{cases} 1 & n < 10 \\ 2T(n-10) + n & n \geq 10 \end{cases}$$

poi analizzata tramite il Teorema delle Ricorrenze lineari di ordine costante, dichiarando una complessità $T(n) = \Theta(n \cdot 2^n)$. Dall'interpretazione delle descrizioni, mi sembra di capire che l'errore si annida nel considerare tutti i casi base "raggiunti" dalle chiamate ricorsive con costo $\mathcal{O}(n)$; ma il caso base si verifica quando $n < 10$, che è un valore costante. Quindi il ciclo for contenuto nel caso base ha comunque costo costante.

Esercizio A2 – Albero sommato – Punti ≥ 10

L'algoritmo costruisce ricorsivamente un nuovo albero T' con la stessa struttura dell'albero di input T . Per ciascun nodo non nullo T :

- crea un nuovo nodo *new*,
- costruisce ricorsivamente i sottoalberi sommati dei figli sinistro e destro,
- inizializza *new.value* con *t.value* e poi vi aggiunge i valori contenuti nelle radici dei sottoalberi appena costruiti.

Poiché tali valori rappresentano le somme dei rispettivi sottoalberi, al termine *new.value* coincide con la somma dei valori di tutti i nodi del sottoalbero radicato in *t*. Nel caso base $t = \text{nil}$, l'algoritmo restituisce **nil**.

Ogni nodo dell'albero di input viene visitato esattamente una volta e in ciascun nodo svolgiamo un numero costante di operazioni. Pertanto il tempo di esecuzione è $\Theta(n)$, dove n è il numero di nodi di T .

`TREE summedTree(TREE t)`

```
if t == nil then
    return nil
else
    TREE new = Tree()
    new.left = summedTree(t.left)
    new.right = summedTree(t.right)
    new.value = t.value
    if new.left != nil then
        new.value += new.left.value
    if new.right != nil then
        new.value += new.right.value
    return new
```

Esercizio A3 – Due caratteri – Punti ≥ 12

Prima soluzione ($\Theta(n^3)$). La soluzione più immediata consiste nell'esaminare tutti i sottovettori contigui $S[start \dots end]$ della stringa. Per ciascun sottovettore si costruisce l'insieme dei caratteri distinti che contiene; se tale insieme ha cardinalità al più 2, si aggiorna una variabile *maxSoFar* con la lunghezza del sottovettore considerato. Poiché il numero di sottovettori è $\Theta(n^2)$ e per ciascuno di essi è necessario scansionare fino a n elementi per determinare i caratteri distinti, il tempo totale richiesto è $\Theta(n^3)$.

```
int maxTwo(item[] S, int n)
{
    int maxSoFar = 0
    for start = 0 to n - 1 do
        for end = start to n - 1 do
            SET chars = Set()
            for i = start to end do
                chars.insert(S[i])
            if chars.size() ≤ 2 then
                maxSoFar = max(maxSoFar, end - start + 1)
        return maxSoFar
}
```

Seconda soluzione ($\mathcal{O}(n^2)$). Un miglioramento naturale consiste nel fissare l'indice di partenza *start* e nell'estendere progressivamente l'indice di fine *end*, mantenendo l'insieme dei caratteri distinti incontrati finora. Quando l'aggiunta di un nuovo carattere provoca la presenza di un terzo carattere distinto, il sottovettore non può più essere esteso e si passa al valore successivo di *start*. In questo modo si evita di ricostruire da zero l'insieme dei caratteri per ogni coppia (*start*, *end*). La complessità è $\mathcal{O}(n^2)$ nel caso pessimo (tutti caratteri uguali); può scendere a $\Omega(n)$ nel caso di tre caratteri che si alternano, come per esempio nel caso della stringa "abcbabcabc".

```
int maxTwo(item[] S, int n)
{
    int maxSoFar = 0
    for start = 0 to n - 1 do
        SET chars = Set()
        int end = start
        while chars.size() ≤ 2 and end < n do
            chars.insert(S[end])
            if chars.size() ≤ 2 then
                maxSoFar = max(maxSoFar, end - start + 1)
            end = end + 1
        return maxSoFar
}
```

Terza soluzione ($\mathcal{O}(n \log n)$). Il problema può essere risolto mediante la tecnica del divide-et-impera. La funzione `maxTwoRec(S, start, end)` restituisce la lunghezza massima di una sottostringa contigua di $S[start \dots end]$ che contenga al più due caratteri distinti.

Se l'intervallo ha lunghezza al più 2, la risposta è immediata – la lunghezza dell'intervallo. Altrimenti, si divide l'intervallo in due metà in corrispondenza di *mid* e si calcolano ricorsivamente le soluzioni ottimali nella metà sinistra e nella metà destra.

Resta da considerare il caso di una sottostringa che attraversa il taglio tra *mid* e *mid* + 1 (soluzione cross), e che quindi contiene necessariamente i caratteri $S[mid]$ e $S[mid + 1]$. In questo caso si mantiene un insieme *chars* dei caratteri distinti ammessi (al più due) e si calcola la massima estensione contigua attorno al centro. Per gestire correttamente anche il caso in cui $S[mid] = S[mid + 1]$, l'estensione viene calcolata due volte: una espandendo prima verso sinistra e poi verso destra, e una in ordine opposto. La migliore soluzione *cross* è il massimo tra i due risultati.

La risposta per l'intervallo corrente è quindi il massimo tra la soluzione migliore a sinistra, a destra e quella che attraversa il centro.

Il passo di combinazione richiede tempo lineare nella dimensione dell'intervallo, quindi

$$T(n) = 2T(n/2) + \mathcal{O}(n) = \mathcal{O}(n \log n).$$

```
int maxTwo(item [] S, int n)
```

```
    if n == 0 then
```

```
        return 0
```

```
    return maxTwoRec(S, 0, n - 1)
```

```
int maxTwoRec(item [] S, int start, int end)
```

```
    if end - start + 1 ≤ 2 then
```

```
        return end - start + 1
```

```
    else
```

```
        int mid = ⌊(start + end)/2⌋
```

```
        int maxLeft = maxTwoRec(S, start, mid)
```

```
        int maxRight = maxTwoRec(S, mid + 1, end)
```

```
        SET chars = Set()
```

```
        chars.insert(S[mid])
```

```
        chars.insert(S[mid + 1])
```

```
        int i = goingLeft(S, start, mid - 1, chars)
```

```
        int j = goingRight(S, mid + 2, end, chars)
```

```
        int maxGoingLeftFirst = j - i + 1
```

```
        SET chars = Set()
```

```
        chars.insert(S[mid])
```

```
        chars.insert(S[mid + 1])
```

```
        int j = goingRight(S, mid + 2, end, chars)
```

```
        int i = goingLeft(S, start, mid - 1, chars)
```

```
        int maxGoingRightFirst = j - i + 1
```

```
        return max(maxLeft, maxRight, maxGoingLeftFirst, maxGoingRightFirst)
```

```
int goingRight(item [] S, int first, int end, SET chars)
```

```
    int j = first
```

```
    while j ≤ end and (chars.size() == 1 or chars.contains(S[j])) do
```

```
        chars.insert(S[j])
```

```
        j = j + 1
```

```
    return j - 1
```

```
int goingLeft(item [] S, int start, int first, SET chars)
```

```
    int i = first
```

```
    while i ≥ start and (chars.size() == 1 or chars.contains(S[i])) do
```

```
        chars.insert(S[i])
```

```
        i = i - 1
```

```
    return i + 1
```

Quarta soluzione ($\mathcal{O}(n)$). È possibile risolvere il problema in tempo lineare tramite la tecnica della *finestra scorrevole* (*sliding window*). Manteniamo due indici *start* e *end* che delimitano una sottostringa $S[start \dots end]$ contenente al più due caratteri distinti. Estendiamo progressivamente *end* verso destra; quando l'aggiunta di $S[end]$ introduce un terzo carattere distinto, spostiamo *start* verso destra

finché nella finestra non rimangono di nuovo al più due caratteri distinti. Durante l'esecuzione manteniamo l'insieme (o dizionario) dei caratteri presenti nella finestra e il loro numero di occorrenze, così da poter rimuovere un carattere quando il suo conteggio scende a zero. La variabile *maxSoFar* viene aggiornata con la massima lunghezza osservata.

```

int maxTwo(item [] S, int n)
    int start = 0
    int maxSoFar = 0
    HASH count = Hash()                                % Dizionario: carattere → occorrenze nella finestra
    for end = 0 to n - 1 do
        if count.lookup(S[end]) == nil then
            | count.insert(S[end], 1)
        else
            | count.insert(S[end], count.lookup(S[end]) + 1)
        while count.size() > 2 do
            int c = count.lookup(S[start])
            if c == 1 then
                | count.remove(S[start])
            else
                | count.insert(S[start], c - 1)
            start = start + 1
        maxSoFar = max(maxSoFar, end - start + 1)
    return maxSoFar

```

Ogni indice *end* avanza da 0 a $n - 1$ una sola volta. Anche *start* avanza monotonamente e non supera mai n ; quindi ogni posizione viene rimossa dalla finestra al più una volta. Il tempo totale è $\mathcal{O}(n)$.

Soluzioni errate Una soluzione errata ricorrente è quella di utilizzare un semplice insieme per considerare i caratteri diversi presenti, e cancellarlo quando si trova il terzo carattere diverso da quello precedente. Qualcosa del genere:

```

int maxTwo(item [] S, int n)
    int maxSoFar = 0
    int count = 0
    SET letters = Set()                                % Insieme: caratteri diversi
    for i = 0 to n - 1 do
        if letters.contains(S[i]) then
            | count = count + 1
            | maxSoFar = max(maxSoFar, count)
        else
            if letters.size() == 2 then
                | count = 0
                | letters = Hash()
            count = count + 1
            maxSoFar = max(maxSoFar, count)
            letters = insert(S[i])
    return maxSoFar

```

Questo però non funziona con stringhe tipo "abccc", perché restituisce 3 (derivante dalle 3 "c") invece che 4 (derivante da "bccc"). Il reset elimina anche il secondo carattere che potrebbe ancora far parte della finestra ottima, quindi non mantiene una sliding window corretta.

Esercizio B1 – New York Dreaming – Punti ≥ 8

Il problema richiede di enumerare tutti i cammini chiusi che partono da s , ritornano in s , non ripetono nodi e hanno lunghezza totale compresa in un intervallo fissato. La soluzione naturale consiste in una visita in profondità (DFS) con backtracking, che costruisce progressivamente il cammino a partire da s , tenendo traccia:

- della distanza mancante;
- dell'insieme dei nodi già visitati, per evitare ripetizioni.

L'algoritmo principale inizializza le strutture ausiliarie e invoca una procedura ricorsiva che esplora tutti i cammini possibili.

```
printPaths(GRAPH  $G$ , NODE  $s$ , int  $k$ )
```

```
  precondition:  $s \in G.V()$ 
```

```
  SET  $visited$  = Set()
```

```
  STACK  $path$  = Stack()
```

```
   $visited.insert(s)$ 
```

```
   $path.push(s)$ 
```

```
  ppRec( $G, s, s, k, visited, path$ )
```

```
ppRec(GRAPH  $G$ , NODE  $u$ , NODE  $s$ , int  $missingDist$ , SET  $visited$ , STACK  $path$ )
```

```
  foreach  $v \in G.adj(u)$  do
```

```
    if  $v == s$  then
```

```
      if  $path.size() > 1$  and  $-500 \leq missingDist - w(u, s) \leq 500$  then
```

```
         $path.push(s)$ 
```

```
        print  $path$ 
```

```
         $path.pop()$ 
```

```
    else
```

```
      if ( $v \notin visited$ ) and  $missingDist - w(u, v) \geq -500$  then
```

```
         $visited.insert(v)$ 
```

```
         $path.push(v)$ 
```

```
        ppRec( $G, v, s, missingDist - w(u, v), visited, path$ )
```

```
         $path.pop()$ 
```

```
         $visited.remove(v)$ 
```

L'algoritmo esplora, tramite DFS con backtracking, tutti i cammini che partono da s e tenta di richiuderli su s rispettando i vincoli di semplicità (nessuna ripetizione di nodi) e di lunghezza.

- La variabile $missingDist$ rappresenta la *distanza mancante* rispetto al valore obiettivo k : inizialmente $missingDist = k$ e, quando percorriamo un arco (u, v) di peso $w(u, v)$, aggiorniamo $missingDist \leftarrow missingDist - w(u, v)$. In questo modo, quando torniamo a s un cammino è accettabile se la distanza residua cade nell'intervallo $[-500, 500]$, cioè se la lunghezza totale è in $[k - 500, k + 500]$.
- L'insieme $visited$ contiene i nodi già presenti nel cammino corrente e impedisce di visitarli di nuovo, garantendo che il cammino costruito sia semplice.
- Lo stack $path$ memorizza la sequenza dei nodi del cammino corrente, e viene aggiornato con operazioni di **push/pop** in modo coerente con il backtracking.

La procedura **printPaths** inizializza $visited$ e $path$ inserendo s come nodo di partenza, e invoca la ricorsione **ppRec** a partire da $u = s$ con distanza residua $missingDist = k$.

La procedura ricorsiva **ppRec** considera ogni arco uscente (u, v) :

- Se $v = s$, l'arco corrente permetterebbe di chiudere il cammino. In tal caso non si prosegue ricorsivamente: si verifica direttamente se la lunghezza del cammino chiuso è nel range, controllando che $-500 \leq \text{missingDist} - w(u, s) \leq 500$. Se sì, si aggiunge temporaneamente s a path , si stampa il cammino, e si ripristina lo stack con una `pop`. Il test $\text{path.size()} > 1$ evita di considerare un cammino banale (senza archi).
- Se $v \neq s$, si può estendere il cammino solo se v non è già stato visitato e se la distanza residua non scende sotto -500 , cioè se $\text{missingDist} - w(u, v) \geq -500$. Quest'ultima condizione implementa una potatura: poiché i pesi rappresentano lunghezze (quindi sono non negativi), una volta che $\text{missingDist} < -500$ non è più possibile rientrare nell'intervallo valido. Quando l'estensione è ammessa, si inserisce v in visited , si fa push su path , si ricorre su v con distanza aggiornata, e infine si annullano le modifiche (pop e rimozione da visited) per esplorare altri rami.

L'algoritmo consente di raggiungere s esclusivamente come *ultimo passo*: s resta in visited per tutta l'esecuzione, e il caso $v = s$ viene gestito con stampa immediata senza ricorsione. In questo modo il cammino visita s solo in posizione iniziale e finale.

Nel caso peggiore, l'algoritmo enumera tutti i cammini chiusi semplici che partono da s e ritornano in s , potenzialmente $(n - 1)!$. La complessità è quindi $\mathcal{O}(n \cdot n!)$.

Esercizio B2 – Personale ospedaliero – Punti ≥ 10

Il problema può essere modellato come un problema di flusso massimo su un grafo bipartito, in cui gli operatori (infermieri e OSS) vengono assegnati ai reparti. Per ogni reparto sono imposti: (i) un numero *esatto* di operatori assegnati, e (ii) un vincolo minimo sul numero di infermieri e sul numero di OSS.

Osservazione preliminare. Se $\sum_{j=1}^m m_j \neq n$, allora non può esistere alcuna assegnazione: ogni operatore deve essere assegnato a un reparto e ogni reparto deve ricevere esattamente m_j operatori.

Costruzione della rete. Costruiamo una rete di flusso con i seguenti nodi:

- una sorgente s e un pozzo t ;
- un nodo i_h per ogni infermiere $p_h \in P_I$;
- un nodo o_h per ogni OSS $p_h \in P_O$;
- per ogni reparto r_j , due nodi rI_j e rO_j , che rappresentano rispettivamente gli infermieri e gli OSS assegnati al reparto r_j ;
- un nodo r_j per ogni reparto.

Gli archi della rete sono definiti come segue:

- un arco (s, i_h) con capacità 1 per ogni infermiere e un arco (s, o_h) con capacità 1 per ogni OSS: ogni operatore può essere assegnato ad al più un reparto;
- per ogni infermiere i_h e reparto $r_j \in \text{capable}_h$, un arco (i_h, rI_j) con capacità 1;
- per ogni OSS o_h e reparto $r_j \in \text{capable}_h$, un arco (o_h, rO_j) con capacità 1;
- per ogni reparto r_j , un arco (rI_j, r_j) con capacità $m_j - m_{O_j}$;
- per ogni reparto r_j , un arco (rO_j, r_j) con capacità $m_j - m_{I_j}$;
- per ogni reparto r_j , un arco (r_j, t) con capacità m_j .

Correttezza (intuizione). Ogni unità di flusso rappresenta l'assegnamento di un operatore a un reparto.

- Le capacità 1 sugli archi $(s, \cdot)$ garantiscono che ciascun operatore sia assegnato ad al più un reparto.
- Gli archi dagli operatori ai nodi rI_j o rO_j esistono solo se il reparto è in *capable*, quindi ogni assegnamento rispetta la qualificazione dell'operatore.
- L'arco (r_j, t) con capacità m_j limita a m_j il numero totale di operatori assegnati al reparto r_j . Se il flusso massimo ha valore $n = \sum_j m_j$, allora necessariamente tutti questi archi sono saturati e ogni reparto riceve *esattamente* m_j operatori.
- L'arco (rI_j, r_j) con capacità $m_j - mo_j$ impone che al più $m_j - mo_j$ infermieri possano essere assegnati a r_j . Poiché il totale in r_j è m_j , segue che il numero di OSS assegnati a r_j è almeno mo_j .
- Simmetricamente, l'arco (rO_j, r_j) con capacità $m_j - mi_j$ implica che al più $m_j - mi_j$ OSS possono essere assegnati a r_j , quindi (dato il totale m_j) gli infermieri assegnati a r_j sono almeno mi_j .

Ne segue che esiste un'assegnazione valida se e solo se il flusso massimo ha valore n (equivalentemente, se satura tutti gli archi $(s, \cdot)$ e tutti gli archi (r_j, t)).

Complessità. Il numero di nodi è

$$|V| = 2 + n + 3m = \mathcal{O}(n + m)$$

(sorgente e pozzo, n nodi operatore, e per ogni reparto i nodi rI_j , rO_j e r_j).

Gli archi sono:

- n archi dalla sorgente agli operatori;
- un arco da ciascun operatore ai reparti per cui è capace: in totale $\sum_i |capable_i|$ (al più $\mathcal{O}(nm)$);
- $2m$ archi dai nodi rI_j, rO_j ai nodi r_j ;
- m archi dai nodi r_j al pozzo.

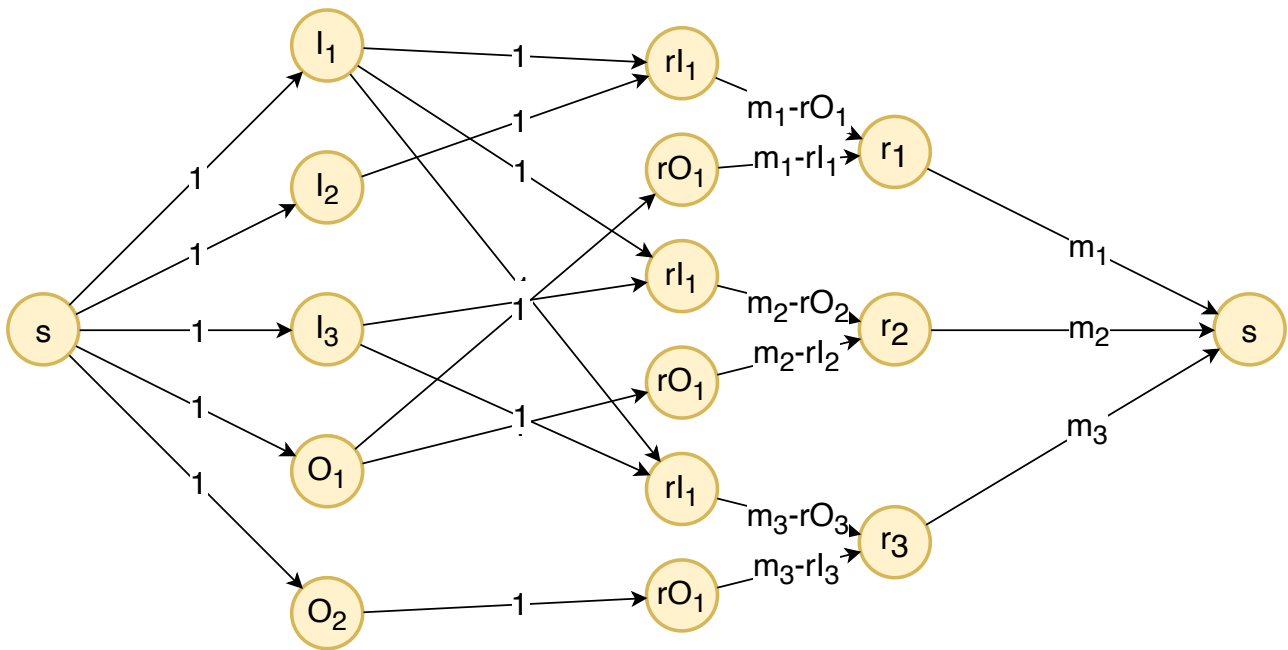
Quindi $|E| = \mathcal{O}(n + \sum_i |capable_i| + m)$ (nel caso peggiore $\mathcal{O}(nm)$).

Il valore del flusso massimo è al più n . Usando Ford–Fulkerson, si ottiene

$$\mathcal{O}(|f^*|(|V| + |E|)) = \mathcal{O}(n(n + m + nm)) = \mathcal{O}(n^2m)$$

nel caso peggiore.

Un esempio di rete di flusso con infermieri, OSS e vincoli per reparto è mostrato in Figura.



Esercizio B3 – Hateville PNRR – Punti ≥ 12

Il vincolo riguarda solo case adiacenti, quindi la scelta del colore per la casa i dipende unicamente dal colore scelto per la casa $i - 1$. Questo suggerisce programmazione dinamica.

Definiamo $DP[h][c]$ il costo minimo per ristrutturare le case $0 \dots h$, dipingendo la casa h col colore c . Allora:

$$DP[h][c] = \begin{cases} cost[0][c] & h = 0 \\ cost[h][c] + \min_{p \neq c} DP[h-1][p] & h > 0 \end{cases}$$

La risposta si trova nella riga $n - 1$ della matrice DP , scegliendo il valore minimo fra tutti i colori possibili.

```
int hateville(int[][] cost, int n, int k)
```

```
    precondition:  $n \geq 0$  and  $k \geq 2$ 
```

```
    if  $n == 0$  then
```

```
        return 0
```

```
    int[][] DP = newint[0...n-1][0...k-1]
```

```
    for  $c = 0$  to  $k - 1$  do
```

```
        DP[0][c] = cost[0][c]
```

```
    for  $h = 1$  to  $n - 1$  do
```

```
        for  $c = 0$  to  $k - 1$  do
```

```
            DP[h][c] =  $+\infty$ 
```

```
            for  $p = 0$  to  $k - 1$  do
```

```
                if  $p \neq c$  then
```

```
                    DP[h][c] = min(DP[h][c], DP[h-1][p] + cost[h][c])
```

```
    return min(DP[n-1])
```

La complessità è $\mathcal{O}(nk^2)$, per via dei cicli annidati.

Soluzione $\mathcal{O}(nk)$ In realtà, la scelta del colore migliore può essere realizzata memorizzando, per ogni casa h , il costo minimo e il secondo costo minimo fra tutti i colori possibili della riga precedente. Se il colore corrente è diverso dal colore che realizza il minimo, si può usare il minimo; altrimenti bisogna usare il secondo minimo.

int hatevilleFast(**int**[][] *cost*, **int** *n*, **int** *k*)

precondition: $n \geq 0$ and $k \geq 2$ **if** $n == 0$ **then** **return** 0**int** *firstCost* = $+\infty$ **int** *secondCost* = $+\infty$ **int** *firstColor* = -1**for** $c = 0$ **to** $k - 1$ **do** **if** $cost[0][c] < firstCost$ **then** *secondCost* = *firstCost* *firstCost* = $cost[0][c]$ *firstColor* = c **else if** $cost[0][c] < secondCost$ **then** *secondCost* = $cost[0][c]$ **for** $h = 1$ **to** $n - 1$ **do** **int** *newFirstCost* = $+\infty$ **int** *newSecondCost* = $+\infty$ **int** *newFirstColor* = -1 **for** $c = 0$ **to** $k - 1$ **do** **int** *bestPrevious* = **iif**($c \neq firstColor$, *firstCost*, *secondCost*) **int** *value* = *bestPrevious* + $cost[h][c]$ **if** $value < newFirstCost$ **then** *newSecondCost* = *newFirstCost* *newFirstCost* = *value* *newFirstColor* = c **else if** $value < newSecondCost$ **then** *newSecondCost* = *value* *firstCost* = *newFirstCost* *secondCost* = *newSecondCost* *firstColor* = *newFirstColor***return** *firstCost*

L'algoritmo mantiene esattamente le informazioni necessarie per calcolare la riga successiva della tabella *DP*, senza materializzare tutta la matrice. Ogni casa richiede una scansione dei k colori, quindi la complessità temporale è $\mathcal{O}(nk)$ e lo spazio aggiuntivo è $\mathcal{O}(1)$.